

ALGORITHMS LAB RECORD

Experiment 1: Merge Sort Performance Analysis

Aim:

To analyze the performance of Merge Sort by executing it for different input sizes and recording the execution time for best, average, and worst cases.

Apparatus:

Personal Computer, Operating System (Windows/Linux/macOS), Python 3 Compiler/Interpreter.

Code:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        L = arr[:mid]
        R = arr[mid:]

        merge_sort(L)
        merge_sort(R)

        i = j = k = 0
        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                arr[k] = L[i]
                i += 1
            else:
                arr[k] = R[j]
                j += 1
            k += 1

        while i < len(L):
            arr[k] = L[i]
            i += 1
            k += 1

        while j < len(R):
            arr[k] = R[j]
            j += 1
            k += 1
```

Observation:

When executing Merge Sort across varying dataset sizes (e.g., $N = 10^3, 10^4, 10^5$), the recorded execution times for the best, average, and worst cases remain nearly identical. The growth rate is super-linear, indicating $O(N \log N)$ complexity.

Result:

The divide and conquer approach ensures consistent $O(N \log N)$ performance across varying datasets. The array division always takes $\log_2(N)$ steps, and the merging phase takes at most N comparisons, regardless of initial data arrangement.

Experiment 2: Quick Sort Pivot Selection Strategies

Aim:

To evaluate the effectiveness of different pivot selection strategies in Quick Sort (first, last, and random) and compare their execution times under different input conditions.

Apparatus:

Personal Computer, Operating System (Windows/Linux/macOS), Python 3 Compiler/Interpreter.

Code:

```
import random

def quick_sort(arr, low, high):
    if low < high:
        pi = partition_random(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

def partition_random(arr, low, high):
    rand_pivot = random.randint(low, high)
    arr[rand_pivot], arr[high] = arr[high], arr[rand_pivot]
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i = i + 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
```

Observation:

Using the first or last element as a pivot on sorted data causes highly imbalanced partitions, degrading the complexity to $O(N^2)$. Selecting a random pivot mitigates this risk, maintaining an expected execution time of $O(N \log N)$ across all inputs.

Result:

The randomized pivot strategy provides optimal performance by effectively breaking pre-existing patterns in the dataset. This ensures balanced splits and avoids worst-case performance while maintaining minimal overhead.

Experiment 3: Binary Search vs. Linear Search

Aim:

To analyze and compare the performance of Binary Search and Linear Search by implementing both algorithms on datasets of varying sizes.

Apparatus:

Personal Computer, Operating System (Windows/Linux/macOS), Python 3 Compiler/Interpreter.

Code:

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Observation:

Linear Search execution time demonstrates a linear growth trend ($O(N)$). In contrast, Binary Search execution times remain exceptionally flat as dataset sizes scale, reflecting an $O(\log N)$ time complexity.

Result:

Linear Search is suitable for small or dynamically changing unsorted structures. Binary Search is highly justified and computationally efficient for large, static, and frequently queried datasets, provided the data is sorted beforehand.

Experiment 4: Matrix Multiplication - Standard vs. Strassen's

Aim:

To evaluate the performance differences between Strassen's Matrix Multiplication and the standard matrix multiplication method.

Apparatus:

Personal Computer, Operating System (Windows/Linux/macOS), Python 3 Compiler/Interpreter.

Code:

```
def standard_matrix_mult(A, B):
    n = len(A)
    C = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

# Note: Strassen's full implementation is highly verbose and omitted for
brevity.
```

Observation:

Standard multiplication executes in strict $O(N^3)$ time. Strassen's method reduces recursive multiplications from 8 to 7, resulting in an $O(N^{2.81})$ complexity. However, standard multiplication runs faster for small matrices ($N < 64$) due to recursion and addition overhead in Strassen's algorithm.

Result:

Strassen's method provides computational advantages primarily for significantly large matrices. In practical applications, a hybrid approach is preferred, using Strassen's until a size threshold is reached, then switching to standard multiplication.

Experiment 5: Trade-offs between Merge Sort and Quick Sort

Aim:

To analyze the trade-offs between Merge Sort and Quick Sort by comparing their execution time, space complexity, and stability.

Apparatus:

Personal Computer, Operating System (Windows/Linux/macOS), Python 3 Compiler/Interpreter.

Code:

```
# (Refer to Experiment 1 for Merge Sort and Experiment 2 for Quick Sort  
implementations)
```

Observation:

Merge Sort guarantees $O(N \log N)$ time and requires $O(N)$ additional memory, but operates stably. Quick Sort averages $O(N \log N)$ time, is often faster in practice due to cache locality, operates in-place requiring only $O(\log N)$ stack space, but is unstable.

Result:

Quick Sort is generally preferred for large-scale array sorting due to average-case speed and low memory footprint. Merge Sort is the justified choice when stability is explicitly required or when processing sequential external storage structures like linked lists.